

For example, in [Figure 14-2](#), the left-hand graph's vertex B is connected to four other vertices, whereas D is connected to only one.

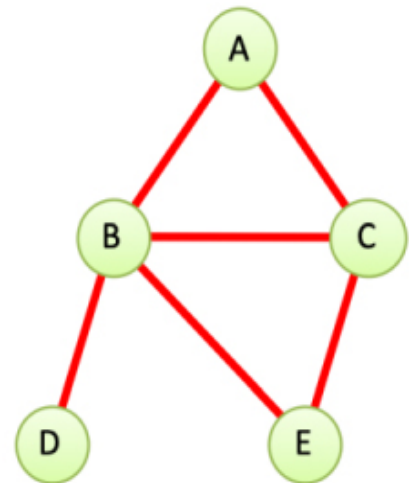
To model this sort of free-form organization, a different approach to representing edges is preferable to that used for trees. Several methods are commonly used for graphs. We examine two (although some might call it three): the *adjacency matrix* and the *adjacency list*. Remember that one vertex is said to be *adjacent* to another if they're connected by a single edge, not a path through several edges.

The Adjacency Matrix

An **adjacency matrix** can be represented as a two-dimensional array in which the elements indicate whether an edge is present between two vertices. If a graph has N vertices, the adjacency matrix is an N×N array. [Table 14-1](#) shows an example of an adjacency matrix for the left-hand graph of [Figure 14-2](#), repeated here.

Table 14-1 *Adjacency Matrix*

	A	B	C	D	E
A	0	1	1	0	0
B	1	0	1	1	1
C	1	1	0	0	1
D	0	1	0	0	0
E	0	1	1	0	0



The vertex labels are used as headings for both rows and columns. An edge between two vertices is indicated by a 1; the absence of an edge is a 0. (You could also use Boolean `True/False` values.) As you can see, vertex B is adjacent to all four other vertices; A is adjacent to B and C; C is adjacent to A, B, and E; D is adjacent only to B; and E is adjacent to B and C. In this example, the “connection” of a vertex to itself is indicated by 0, so the diagonal from upper left to lower right, A-A to E-E, which is called the **identity diagonal** and shaded blue, is all 0s. The entries on the identity diagonal don't

convey any real information, so you can equally well put 1s along it, if that's more convenient to a program. (A kind of graph called a *pseudograph* allows edges that go from a vertex to itself. Such graphs can use the diagonal of the adjacency matrix to represent the presence or absence of such edges.)

Note that the triangular-shaped part of the matrix above the identity diagonal is a mirror image of the part below; both triangles contain the same information. This redundancy is a bit inefficient, but there's no simple way to create a triangular array in most computer languages, so it's simpler to accept the redundancy. Consequently, when you add an edge to the graph, you make two entries in the adjacency matrix rather than one.

Note, in this chapter, we focus on unweighted graphs, and the edges don't need separate labels like in the bridges of Königsberg. Graphs that allow multiple edges between a single pair of vertices are called *multigraphs*. They can be quite useful as in the case of the bridges of Königsberg but are beyond the scope of this text.

Using Hash Tables for the Adjacency Matrix

One way to improve storage efficiency is to keep the matrix as a hash table rather than a two-dimensional array. To do this, the hash table must accept keys with two parts—one for each vertex. That's straightforward, as discussed in [Chapter 11](#), “[Hash Tables](#).” The individual characters in a string or the indices in a tuple can be hashed using different weights in the hashing function to produce a single hash index.

To represent an edge using a hash table adjacency matrix, you make an entry at a particular pair of vertices. For example, to add an edge between vertices 2 and 7, you would insert a value for the key `(2, 7)`. Using the data structures from [Chapter 11](#), you could write

```
adjacencyMatrix = HashTable()
adjacencyMatrix.insert((2, 7), True)
```

After the edges are placed in the matrix, programs can determine whether two edges are adjacent by using the hash table's `search()` method. For example, to see whether two vertices are adjacent, you would evaluate `adjacencyMatrix.search((4, 12))`. If no entry had been made for `(4, 12)`, the search would return `None`, which Python treats as `False` in a Boolean context. If an entry had been made for that key, its `True` value would be returned.

With either hash tables or two-dimensional arrays, you can avoid the duplicate storage of the two triangular halves of the matrix by always using the smaller vertex index in the first position. In other words, to check if vertex 19 is adjacent to vertex 8 by a bidirectional edge, you would check the matrix at (8, 19) instead of (19, 8). Reordering the vertex indices would be necessary for all operations on the matrix—insertion, deletion, and searching. That adds a little extra time to each of those operations. The alternative of adding time to the insertion and deletion operations by using both orderings and updating both matrix cells is usually preferable when searching will be much more frequent than inserting and deleting. It also is better for directional graphs, as you see later.

Hash tables have an extra benefit over two-dimensional arrays when there are few edges. If a graph has, for example, a thousand vertices and two thousand edges, the two-dimensional array needs storage for a million cells while the hash table needs only enough for the two thousand edges (so perhaps four thousand cells). The difference becomes significant for large graphs because the memory needed for two-dimensional arrays is $O(N^2)$, where N is the number of vertices, limiting what can be kept in the computer's memory.

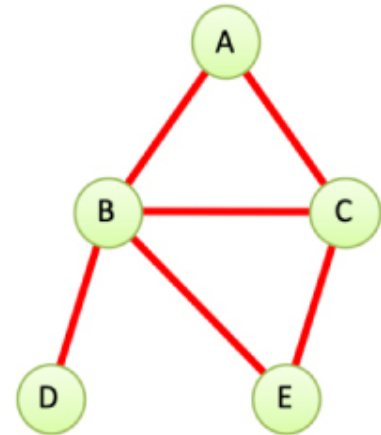
On the other hand, two-dimensional arrays are easier to decompose into rows and columns. Many graph algorithms were developed to take advantage of the capability to access rows and columns quickly. The choice of representation depends on what operations need to be done quickly on the graph.

The Adjacency List

The other common way to represent edges is with a list. The list in **adjacency list** refers to a linked list of the kind examined in [Chapter 5](#), “[Linked Lists](#).” In actuality, an adjacency list is an array of lists, or sometimes a list of lists, or sometimes a list stored in each `Vertex`. Each individual list contains references to the adjacent vertices of that vertex. [Table 14-2](#) shows the adjacency lists for the left-hand graph of [Figure 14-2](#), repeated here.

Table 14-2 *Adjacency Lists*

Vertex	Adjacent Vertices List
A	B→C
B	A→C→D→E
C	A→B→E
D	B
E	B→C



In this table, the → symbol indicates a link in a linked list. Each link in the list is a vertex. Here, the vertices are arranged in alphabetical order in each list, although that's not strictly necessary. More likely, the order of the vertices would depend on the order the edges were added to the graph.

Don't confuse the contents of adjacency lists with paths. The adjacency list shows which vertices are adjacent to—one edge away from—a given vertex, not paths from vertex to vertex.

In the next chapter we discuss when to use one of the adjacency matrices as opposed to an adjacency list. The Visualization tool shown in this chapter shows the adjacency matrix approach, but in many cases, the list approach is more efficient.

Adding Vertices and Edges to a Graph

To add a vertex to a graph, you make a new `Vertex` object, insert it into a vertex array or list, and grow the adjacency structure, as we discuss shortly. In a real-world program, a vertex might contain many data elements, but for simplicity of the program examples, you can assume that it contains only a single attribute, its name. (The visualization tool also associates different colors with the vertices in addition to their names to indicate the different data they reference.) Thus, the creation of a vertex looks something like this:

```
vertices.append(Vertex('F'))
```

This command inserts a vertex object named F at the end of the Python array called `vertices`.

How you add an edge to a graph depends on whether you're using an adjacency matrix or adjacency lists to represent the graph. Let's say that you're using an adjacency matrix and want to add an edge between vertices 1 and 3. These numbers correspond to the array indices in `vertices` where the vertices are stored.

When the adjacency matrix was created, it would have a particular size. If adding a new vertex means it needs an extra row and column, the array will need to grow. When the adjacency matrix, `adjMat`, is first created, it is filled it with 0s or perhaps a Boolean `False`. When it grows, all the new cells must be filled with that same value. Adjacency matrices represented as hash tables do not require growing when new vertices are added. They grow when edges are added as new keys as described in [Chapter 11](#), “Hash Tables.”

Two-Dimensional Arrays and Python

To insert the edge between vertices 1 and 3 in two-dimensional array in Java or C++, you could write

```
adjMat[1][3] = 1;
adjMat[3][1] = 1;
```

The core Python language supports only one-dimensional arrays, although extensions like NumPy provide multidimensional arrays. You can store an array in the cell of another array to approximate multidimensional arrays. For example, you can create an array of vertices plus an adjacency matrix, and insert an edge with code like the following:

```
vertices = []                                # A list of vertices
vertices.append(Vertex('A'))
vertices.append(Vertex('B'))
vertices.append(Vertex('C'))
vertices.append(Vertex('D'))

adjMat = [ [False for v in range(len(vertices))]
           for _ in range(len(vertices)) ]
adjMat[1][3] = True
adjMat[3][1] = True
```

To create the adjacency matrix, we've used nested list comprehensions that create an outer array filled with arrays. The inner comprehension, `[False for v in range(len(vertices))]`, creates a list/array of four cells filled with the value `False`. The outer comprehension repeats the inner creation four times to

create four cells filled with lists/arrays. The result of the last two assignment statements leaves `adjMat` holding

```
[[False, False, False, False],
 [False, False, False, True],
 [False, False, False, False],
 [False, True, False, False]]
```

The `True` values show up in the rows and columns indexed by (1, 3) and (3, 1). Note that Python's capability to expand a list/array using the multiplication operator doesn't work correctly here. If you try

```
badMat = [ [False] * len(vertices) ] * len(vertices)
badMat[1][3] = True
```

the resulting `badMat` array contains

```
[[False, False, False, True],
 [False, False, False, True],
 [False, False, False, True],
 [False, False, False, True]]
```

That's not what you want, but why did it come out that way? The reason is that the multiplication operator fills the cells of the expanded list with the *exact same value*. The inner multiply creates a four-cell array of `False` values. The outer multiply does the same kind of expansion, and every cell is filled with a reference to the *same* inner four-cell array. Because the same inner array is shared among all the cells of the outer array, assigning `True` to cell 3 in one row affects all the rows.

Storing Edges in Adjacency Lists

Returning to the concept of using adjacency lists, they have a similar nested list/array structure, but with a significant difference. Instead of each row having the same length, they are variable-length lists of vertex indices. For example,

```
adjList = [ [] for v in range(len(vertices)) ]
adjList[1].append(3)
adjList[3].append(1)
produces an array containing
```

```
[[], [3], [], [1]]
```

The cells of this array contain lists of vertex indices. Cell 0 has the (empty) list of vertices for vertex 0, and so on. The complete adjacency list shows that

vertices 0 and 2 have no edges, whereas vertices 1 and 3 share an edge.

Note that the multiplication operator also fails to work for adjacency lists too. For example,

```
badList = [[]] * len(vertices)
badList[1].append(3)
```

produces

```
[[3], [3], [3], [3]]
```

The list comprehension method effectively executes a loop to create separate elements for its output array; that's how it builds separate lists on each iteration.

The Graph Class

Let's make this discussion more concrete with a Python `Graph` class that constructs a vertex list and an adjacency matrix and contains methods for adding vertices and edges. [Listing 14-2](#) shows the code.

Listing 14-2 *The Basic Graph Class*

```
class Graph(object):
    # A graph containing vertices and edges
    def __init__(self):
        # Constructor
        self._vertices = []
        # A list/array of vertices
        self._adjMat = {}
        # A hash table mapping vertex pairs to 1

    def nVertices(self):
        # Get the number of graph vertices, i.e.
        return len(self._vertices) # the length of the vertices list

    def nEdges(self):
        # Get the number of graph edges by
        return len(self._adjMat) // 2 # dividing the # of keys by 2

    def addVertex(self, vertex):
        # Add a new vertex to the graph
        self._vertices.append(vertex) # Place at end of vertex list

    def validIndex(self, n):
        # Check that n is a valid vertex index
        if n < 0 or self.nVertices() <= n:
            # If it lies outside the
            raise IndexError # valid range, raise an exception
        return True # Otherwise it's valid

    def getVertex(self, n):
        # Get the nth vertex in the graph
        if self.validIndex(n):
            # Check that n is a valid vertex index
```

```

        return self._vertices[n] # and return nth vertex

    def addEdge(self, A, B): # Add an edge between two vertices A & B
        self.validIndex(A)   # Check that vertex A is valid
        self.validIndex(B)   # Check that vertex B is valid
        if A == B:           # If vertices are the same
            raise ValueError  # raise exception
        self._adjMat[A, B] = 1 # Add edge in one direction and
        self._adjMat[B, A] = 1 # the reverse direction

    def hasEdge(self, A, B): # Check for edge between vertices A & B
        self.validIndex(A)   # Check that vertex A is valid
        self.validIndex(B)   # Check that vertex B is valid
        return self._adjMat.get( # Look in adjacency matrix hash table
            (A, B), False)      # Return either the edge count or False

```

This implementation makes use of Python’s `list` type to manage the list of vertices and the `dict` type to store the adjacency matrix. These two structures behave like the `Stack` and `HashTable` classes you saw in earlier chapters, but with some different syntax. The constructor for the `Graph` creates an empty `list` called `_vertices` and an empty `dict` called `_adjMat`. You saw hash tables used to store a grid of cells in [Chapter 12, “Spatial Data Structures.”](#) In case you skipped that, we describe the use of Python’s `dict` type as a dictionary (hash table) in more detail [here](#).

The pair of curly braces, `{}`, in the constructor creates an empty hash table that can accept most Python data structures as a key. For this graph, we use tuples of vertex indices, like `(2, 7)`, as the keys to the adjacency matrix. When we need to store a 1 in the adjacency matrix for a particular tuple, we could write

```
self._adjMat[(2, 7)] = 1
```

The square brackets after `_adjMat` surround the key to the hash table. That tells Python that it should hash the key inside to find where to place the value in the hash table. Later we can retrieve the value from the hash table using the same syntax.

You can also use the slightly simpler syntax

```
self._adjMat[2, 7] = 1
```

to do the same thing. To programmers familiar with other languages, this syntax might look like a multidimensional array reference, but it is not. The comma in the expression within the brackets tells Python to construct a **tuple**.

In this case, it constructs the tuple `(2, 7)` and uses that as the key for the hash table. The hashing function uses all the tuple elements in calculating the hash table index. This means that the value gets stored in a unique location for the key `(2, 7)`. In a two-dimensional array, the cell at row 2, column 7 would be addressed, but in the hash table, some location in its one-dimensional array is used. To the calling program, it doesn't matter which one, as long as that exact same cell is found when it references `(2, 7)` in the hash table later.

The Python syntax for addressing one-dimensional arrays and hash tables is identical. When Python sees `var[2]`, the integer inside the square brackets tells the interpreter that `var` should be accessed as a one-dimensional array. If it sees `var[2, 7]`, the comma inside causes it to construct a tuple. With a tuple as the index (or key), it accesses `var` as a hash table, not an array. If some other data type like a string (for example, `var['2 7']`) or a Boolean (for example, `var[True]`) is inside the brackets, it treats that as a key to a hash table. Only integers inside the brackets cause Python to treat the object as an array.

Moving on to the first method defined in the `Graph` class of [Listing 14-2](#), you find that `nVertices()` makes use of Python's `len()` function to get the length of the list/array holding the vertices. A freshly constructed `Graph` has an empty `_vertices` list, so the length is zero.

The second method, `nEdges()`, is similar but uses the length of the hash table, `_adjMat`, in its calculation. Python uses the number of keys that have been stored in the hash table as its length. Because we plan to store vertex pairs along with their mirror image—for example `(2, 7)` and `(7, 2)`—as separate keys for each edge, the total number of edges is half the number of keys.

The third method, `addVertex()`, adds a vertex to the graph by using Python's built-in `append()` method on the `_vertices` list. This is just like pushing an element on a stack (assuming the top of the stack is the end of the list). We left out a check in this method that the `vertex` argument is one of the `Vertex` objects as defined in [Listing 14-1](#), but that would be good to include.

Next, the program introduces a simple test for a valid vertex index, `validIndex()`. Because callers specify vertices by their index, they could specify indices outside the range of those already added to the graph. This *predicate*—a function with a Boolean result—checks whether the index lies outside of the range `[0, nVertices)`, raising the `IndexError` exception if it does. This is the same exception that Python uses for invalid array indices.

The `getVertex()` method gets a vertex from the graph based on an index, `n`. After the index is validated, the vertex object can be retrieved from the array.

The `addEdge()` method takes two vertex indices, `A` and `B`, as parameters. To create the edge, it first verifies that both `A` and `B` are valid vertex indices. Without these checks, the `Graph` could become internally inconsistent. It also checks whether `A` and `B` are the same index. That would create an edge from a vertex to itself. This simple `Graph` class doesn't allow the creation of pseudographs. Finally, the method updates the adjacency matrix to create the edge. It uses both orders of the vertices because the edge is bidirectional.

We now have a basic `Graph` object that can expand to accept any number of vertices and edges. The next important method is one that tests whether an edge exists between two vertices. The `hasEdge()` method takes two vertex indices, `A` and `B`, as parameters. They are checked for validity like before. With valid indices, the adjacency matrix can be checked to see whether an edge was defined. You might expect to use the same Python syntax, `_adjMat[A, B]`, to test for that edge. That, however, causes a problem if the edge does not exist. Python hash tables raise a `KeyError` exception when asked to access a key that has not been previously inserted.

There are a couple of ways to address missing keys in the adjacency matrix. One is to catch the `KeyError` exception that could occur when accessing `_adjMat[A, B]`. The other is to use Python's `get()` method for hash tables, which is what this implementation does. The `get()` method takes the key as the first parameter plus a second one for the value to return if the key is not in the hash table. We pass the tuple, `(A, B)`, for the key. The default value is set to `False` so that `hasEdge()` returns that when the edge does not exist. If the edge does exist, the call to `get()` will return `1`, the value that was inserted by `addEdge()`, which is interpreted as `True` in Boolean contexts.

Another way to implement the `hasEdge(A, B)` check would be to return the value of this expression `(A, B) in self._adjMat`. Python uses the `in` operator to test whether a key has been inserted in a hash table. Because we only ever set the values in `_adjMat` to `1`, we could ignore the value and only check for the presence of the key. When edges are deleted, however, we could not simply set the value to `0` if we use the `(A, B) in self._adjMat` test; we would need to remove the key from the hash table.

Traversal and Search

One of the most fundamental operations to perform on a graph is finding which vertices can be reached from a specified vertex. This operation is used to find the connected components of a graph and underlies many more complex operations. When we look at a graph diagrams, it's usually immediately obvious which vertices are connected, at least for simple graphs. To the computer, however, it must discover what vertices are connected by chaining together edges. Let's look at some examples of discovering what's connected.

Imagine that you are traveling to a foreign country by airplane or boat for a vacation. When there, you will be visiting the countryside by bicycle, and you want to know all the places you can go. Paved roads and some dirt roads would be good for the trip, but roads that are closed for repair or washed out with mud or floods are not worth traversing. The road conditions might be cataloged somewhere, but in some cases, you would have to go to a nearby location to find out whether the road was passable. Based on the road conditions, some towns could be reached, whereas others couldn't. You still want to have the best vacation possible, so you would determine what places are accessible via bicycle and decide your route among them. You might have to remake the plan as you travel and learn more about road conditions in each area.

Other situations where you need to find all the vertices reachable from a specified vertex are in designing circuits and plumbing networks. Electronic circuits are composed of components like transistors that are connected by conductors such as wires or metal paths. In plumbing networks various components such as water heaters, faucets, drains, and gas stoves connect via pipes. [Figure 14-4](#) shows small examples of an electronic circuit and a plumbing network. In both cases, many of the components are connected, and others are not. It's critical to their function that the connections are complete. If not, hot water might not reach a particular faucet, or the signal from an antenna might not reach a decoder. Perhaps even more important, the disconnected vertices must remain that way—lest the power supply connect directly to a speaker or the gas supply connect to a faucet.

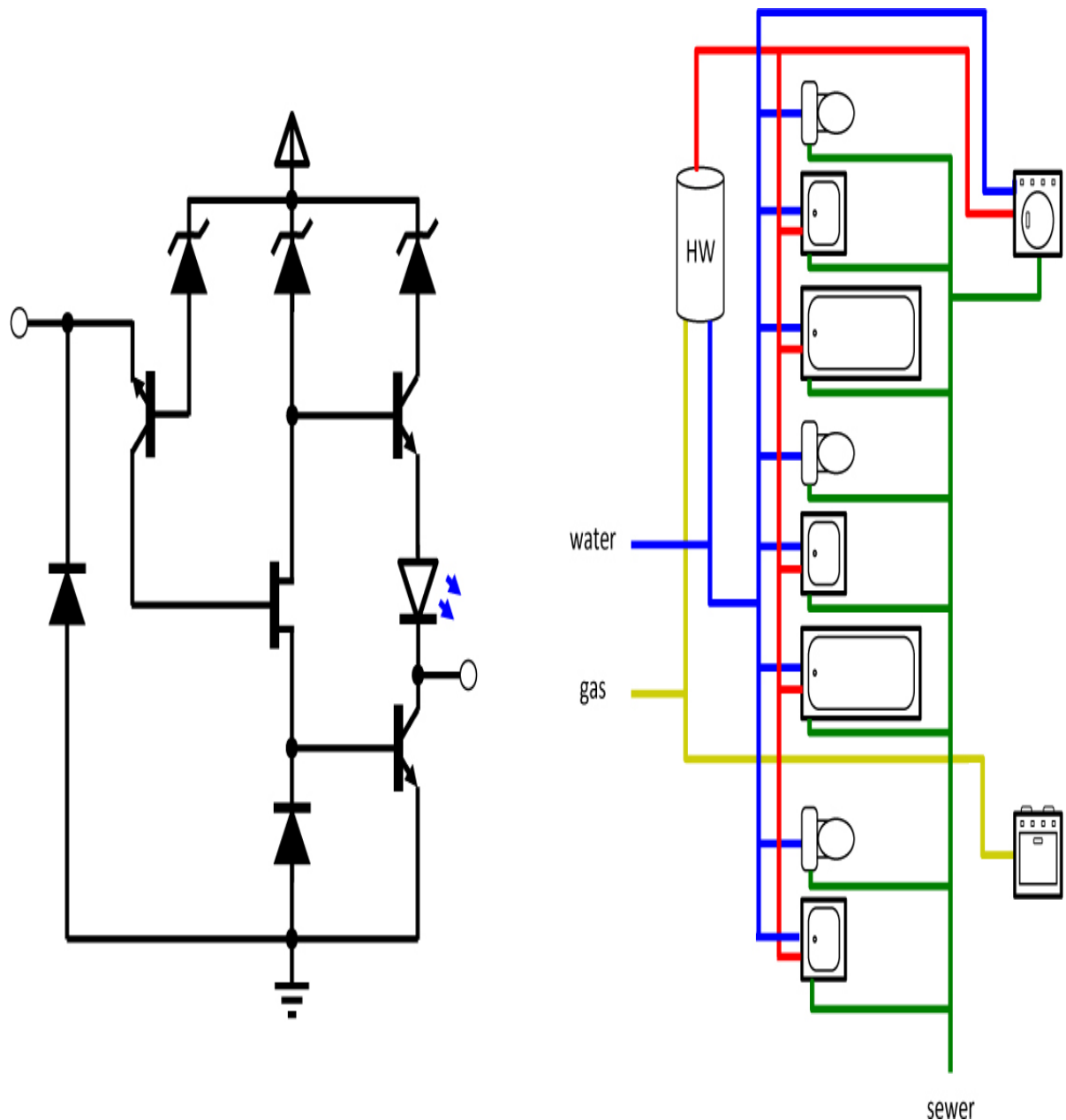


Figure 14-4 *An electronic circuit and a plumbing network*

In these kinds of networks, it's important to find all the vertices connected to a given vertex. That defines the connected components of the graph. Each connected component must be realized in the form of electrical or plumbing connections. That's often easy for you to see in diagrams like [Figure 14-4](#), especially when the colors of the edges in the plumbing diagram clearly separate the kinds of pipes. It's much less clear in the case of the road network for bicycling, especially when you must travel to an area to learn what's connected and what's not.

Assume that you've been given a graph that describes a network. Now you need an algorithm that provides a systematic way to start at a specified vertex and move along edges to other vertices in such a way that, when it's done, you are guaranteed that it has *visited* every vertex that's connected to the starting vertex. Here, as it did in [Chapter 8](#), where we discussed binary trees, *visit* means to perform some operation on the vertex, such as displaying it, adding it to a collection, or updating one of its attributes.

There are two common approaches to traversing a graph: **depth-first** (*DF*) and **breadth-first** (*BF*). Both eventually reach all connected vertices but differ in the order they visit them. The depth-first traversal is implemented with a stack, whereas breadth-first is implemented with a queue. You can traverse all the connected vertices or perhaps stop when you find a particular vertex. When the goal is to stop at a particular vertex, the operation is called **depth-first search** (*DFS*) or **breadth-first search** (*BFS*) instead of traversal.

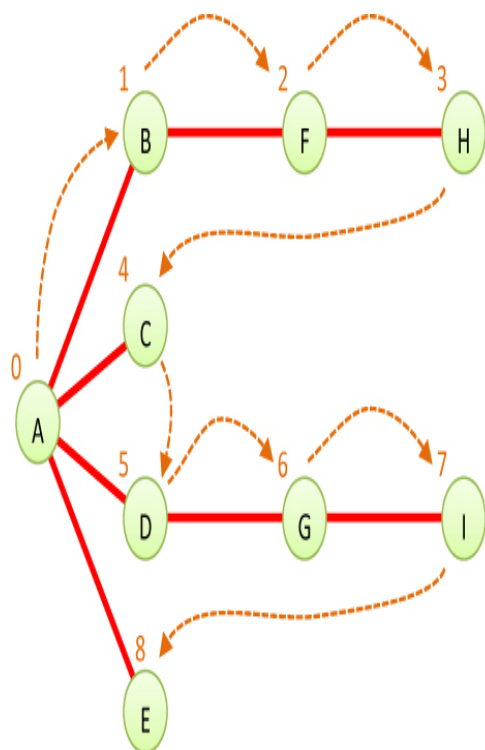
You saw in [Chapter 8](#) that the different traversal orders of binary trees—pre-order, in-order, and post-order—had different properties and uses. The same is true of graphs. The choice of depth-first or breadth-first depends on the goal of the operation.

Depth-First

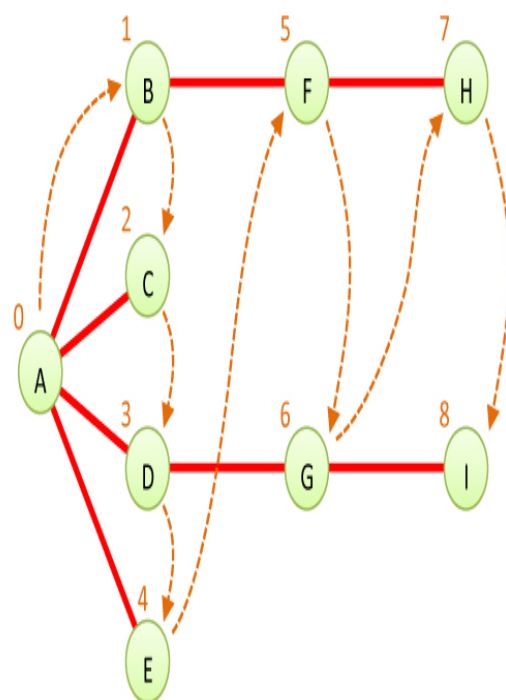
The depth-first traversal uses a stack to remember where it should go when it reaches a dead end (a vertex with no adjacent, unvisited vertices). We show an example here, encourage you to try similar examples with the Graph Visualization tool, and then finally show some code that carries out the traverse.

An Example

Let's look at the idea behind the depth-first traversal in relation to the graph in [Figure 14-5](#). The colored numbers and dashed arrows in this figure show the order in which the vertices are visited (which differ from the ID numbers used to identify vertices).



Depth-First Traversal



Breadth-First Traversal

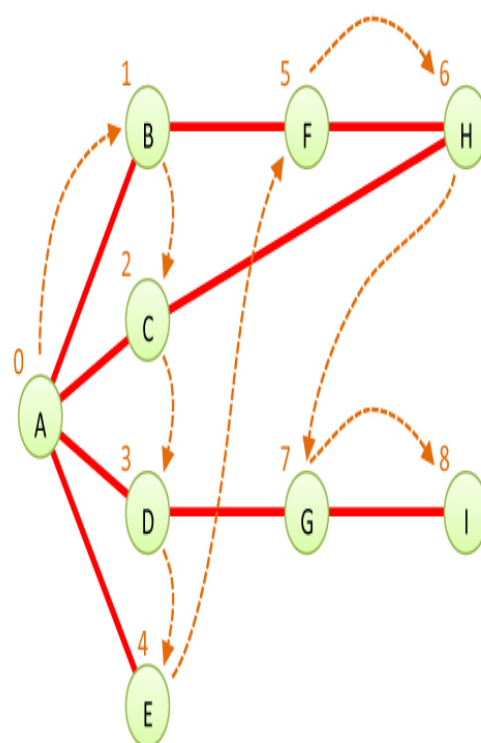
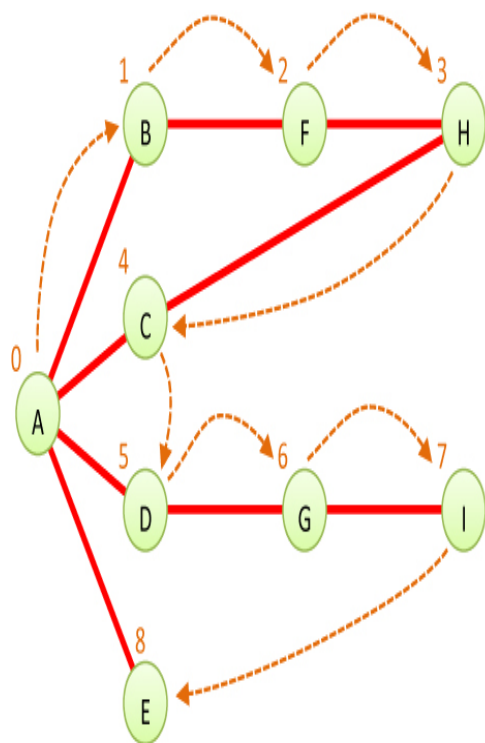


Figure 14-5 *Depth-first traversal example*

To carry out the depth-first traversal, you pick a starting point—in this case, vertex A. You then do three things: visit this vertex, push it onto a stack so that you can remember it, and mark it so that you won't visit it again.

Next, you go to any vertex adjacent to A that hasn't yet been visited. We'll assume the vertices are selected in alphabetical order, so that brings up B. You visit B, mark it, and push it on the stack.

Now what? You're at B, and you do the same thing as before: go to an adjacent vertex that hasn't been visited. This leads you to F. We can call this process Rule 1.

Rule 1

If possible, visit an adjacent unvisited vertex, mark it, and push it on the stack.

Applying Rule 1 again leads you to H. At this point, however, you need to do something else because there are no unvisited vertices adjacent to H. Here's where Rule 2 comes in.

Rule 2

If you can't follow Rule 1, then, if possible, pop a vertex off the stack.

Following Rule 2, you pop H off the stack, which brings you back to F. F has no unvisited adjacent vertices, so you pop it. The same is true of vertex B. Now only A is left on the stack.

A, however, does have unvisited adjacent vertices, so you visit the next one, C. The visit to C shows that it is the end of the line again, so you pop it, and you're back to A. You visit D, G, and I, and then pop them all when you reach the dead end at I. Now you're back to A. You visit E, and again you're back to A.

This time, however, A has no unvisited neighbors, so you pop it off the stack. Now there's nothing left to pop, which brings up Rule 3.

Rule 3

If you can't follow Rule 1 or Rule 2, you're done.

Table 14-3 shows how the stack looks in the various stages of this process, as applied to Figure 14-5. The contents of the stack show the path you took from the starting vertex to get where you are (at the top of the stack). As you move away from the starting vertex, you push vertices as you go. As you move back toward the starting vertex, you pop them. The order in which you visit the vertices is ABFHCDGIE. Note that this is not a path, just a vertex list, because H is not adjacent to C, for example.

Table 14-3 *Stack Contents During Depth-First Traversal*

Event	Stack
Visit A	A
Visit B	AB
Visit F	ABF
Visit H	ABFH
Pop H	ABF
Pop F	AB
Pop B	A
Visit C	AC
Pop C	A
Visit D	AD
Visit G	ADG
Visit I	ADGI
Pop I	ADG
Pop G	AD
Pop D	A
Visit E	AE
Pop E	A
Pop A	
Done	

You might say that the depth-first algorithm likes to get as far away from the starting point as quickly as possible and returns only when it reaches a dead end. If you use the term *depth* to mean the distance from the starting point, you can see where the name *depth-first* comes from.

An Analogy

An analogy you might think about in relation to a depth-first search is a maze. The maze—perhaps one of the people-size ones made of hedges, popular in England, or corn stalks, popular in America—consists of narrow passages (think of edges) and intersections where passages meet (vertices).

Suppose that Minnie is lost in a maze. She knows there’s an exit and plans to traverse the maze systematically to find it. Fortunately, she has a ball of string and a marker pen. She starts at some intersection and goes down a randomly chosen passage, unreeling the string. At the next intersection, she goes down another randomly chosen passage, and so on, until finally she reaches a dead end.

At the dead end, she retraces her path, reeling in the string, until she reaches the previous intersection. Here she marks the path she’s been down, so she won’t take it again, and tries another path. When she’s marked all the paths leading from that intersection, she returns to the previous intersection and repeats the process.

The string represents the stack in depth-first: it “remembers” the path taken to reach a certain point. The pen represents marking vertices as visited. If Minnie didn’t have a pen, she could always choose, say, the leftmost passage to visit when arriving at an intersection. When returning along the string and coming back to an intersection, she would choose the next passage to the right to visit and continue to follow the string back if there were no more passages to the right. The critical thing is that Minnie needs to remember having visited an intersection/vertex so that she doesn’t just revisit them. The sense of left and right doesn’t exist in most graphs, and computers don’t use pens; they must use explicit marking.

The Graph Visualization Tool and Depth-First Traverse

You can try out the depth-first traversal with the Depth-First Traverse button in the Graph Visualization tool. Start the tool (as described in [Appendix A](#), “[Running the Visualizations](#)”). At the beginning, there are no vertices or edges, just an empty shaded rectangle. You create vertices by double-clicking the desired location within the shaded box. The first vertex is automatically labeled A, the second one is B, and so on. They’re each given a different color.

To make an edge, drag the pointer from one vertex to another. [Figure 14-6](#) shows part of the graph of [Figure 14-5](#) as it looks while being created using the tool. The adjacency matrix appears in the lower-right corner. When the visualization tool first starts, there are no vertices, and the matrix is empty. The matrix and the `_vertices` table in the upper right grow as you add vertices.

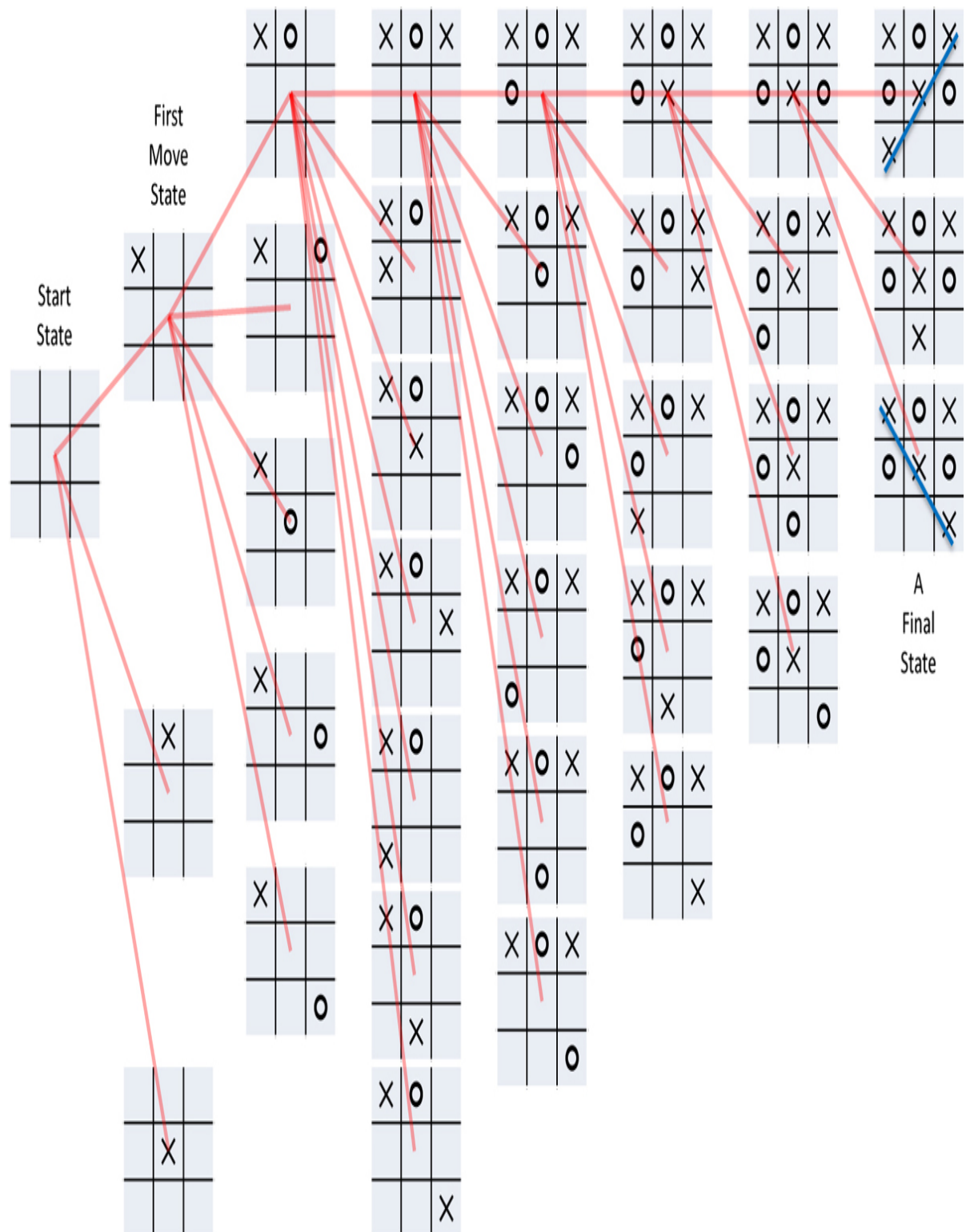


Figure 14-6 *The Graph Visualization tool*

You can edit the graph in many ways. Like the other visualization tools, this one has buttons in the operations area that allow you to create and delete vertices by their label. Because the tool must find vertices by their label, all the names must be unique (and short enough to fit in the little circles and rectangles). When you add a vertex—say with the label V—the program will choose a random location for it within the shaded box and conveniently update the label in the text entry box to W in case you plan to add another. That allows you to click New Vertex N times to create N vertices with unique names. You can also put a number in the text entry box and select the Random Fill button to create that many more vertices.

If you want to rearrange the vertex positions, press and hold the Shift key while you click a vertex and drag it to its new position. The visualization tool allows you to place the vertex only where it lies completely within the shaded box and does not overlap another vertex. If you want to get rid of a vertex or an edge, you can double-click it.

Another way to delete edges is by clicking the box corresponding to that edge in the adjacency matrix table. Because edges are either present or absent in this kind of graph, each click toggles the edge on or off. You can collapse the adjacency matrix by pressing the  button and restore it with the  button. That capability is especially useful for large graphs.

To run the depth-first traversal algorithm, select a starting vertex by clicking it and then select the Depth-First Traverse button. When you click a vertex, the blue circle marks it as the starting vertex for the traversal, as vertex A is marked in [Figure 14-6](#). Try it on the graph shown in [Figure 14-6](#) or one of your own design. The animation shows the creation of a stack below the shaded box and how vertices are pushed on and popped off to traverse the graph. Of course, the traversal covers only the connected component that includes the selected vertex.

Python Code

The depth-first traversal algorithm must find the vertices that are unvisited and adjacent to a specified vertex. How should this be done? The adjacency matrix holds the answer. By going to the row for the specified vertex and stepping across the columns, you can pick out the columns with a 1; the column number is the number of an adjacent vertex. You can then check whether this vertex is unvisited by examining the value for that vertex in separate array called `visited`. If that value is `False`, you've found what you want—the next vertex

to visit. If no vertices on the row are simultaneously 1 (adjacent) and unvisited, there are no unvisited vertices adjacent to the specified vertex. The code for this process is made up of several parts—a generator for all vertices, a generator for getting adjacent vertices, and a generator for getting unvisited vertices—as shown in [Listing 14-3](#).

Listing 14-3 *Code for Traversing Adjacent Vertices*

```
class Graph(object):
...
    def vertices(self):          # Generate sequence of all vertex indices
        return range(self.nVertices()) # Same as range up to nVertices

    def adjacentVertices(      # Generate a sequence of vertex indices
        self, n):             # that are adjacent to vertex n
        self.validIndex(n)     # Check that vertex n is valid
        for j in self.vertices(): # Loop over all other vertices
            if j != n and self.hasEdge(n, j): # If other vertex connects
                yield j           # via edge, yield other vertex index

    def adjacentUnvisitedVertices( # Generate a sequence of vertex
        self, n,                  # indices adjacent to vertex n that do
        visited,                  # not already show up in the visited list
        markVisits=True):         # and mark visits in list, if requested
        for j in self.adjacentVertices(n): # Loop through adjacent
            if not visited[j]: # vertices, check visited
                if markVisits: # flag, and if unvisited, optionally
                    visited[j] = True # mark the visit

                yield j         # and yield the vertex index
```

The generator for all vertex indices is the same as Python's `range()` generator for index values up to `nVertices - 1`. The `adjacentVertices()` generator takes a vertex index, `n`, as the starting vertex. It checks that `n` is within the bounds of the known vertices and then starts a loop over all the other vertex indices. For another vertex, `j`, that is not `n` but does have an edge in the adjacency matrix to `n`, the generator yields vertex `j` for processing by the caller. When all `nVertices` have been checked, the generator is finished, and it raises the `StopIteration` exception.

One way to implement the marking of vertices as visited or unvisited is to share a data structure between the caller and the generator. The simplest approach

uses an array of Boolean flags indicating which of the `nVertices` have been visited.

The `adjacentUnvisitedVertices()` generator of [Listing 14-3](#) takes a starting vertex index, `n`, plus a `visited` array and a `markVisits` flag as parameters. The `visited` array should have at least one cell for all the vertices, initially with all false values. This array can be created in Python using an expression like `[False] * nVertices` or `[None] * nVertices`. When `markVisits` is true, the generator will set the flag for the cells it visits to `True`.

By using `adjacentVertices()` to enumerate the vertices adjacent to vertex `n`, all that remains is the check for whether they have been visited. If they have not, they are optionally marked as visited before yielding them. With this kind of generator, it's easy to define different traversal orderings like depth-first, as shown in [Listing 14-4](#).

Listing 14-4 *Implementation of Depth-First Traversal of a Graph*

```
class Stack(list):          # Use list to define Stack class
    def push(self, item): self.append(item) # push == append
    def peek(self): return self[-1] # Last element is top of stack
    def isEmpty(self): return len(self) == 0

class Graph(object):
    ...
    def depthFirst(
        self, n):          # Traverse the vertices in depth-first
                           # order starting at vertex n
        self.validIndex(n) # Check that vertex n is valid
        visited = [False] * self.nVertices() # Nothing visited initially
        stack = Stack()    # Start with an empty stack
        stack.push(n)      # and push the starting vertex index on it
        visited[n] = True  # Mark vertex n as visited
        yield (n, stack)   # Yield initial vertex and initial path
        while not stack.isEmpty(): # Loop until nothing left on stack
            visit = stack.peek() # Top of stack is vertex being visited
            adj = None
            for j in self.adjacentUnvisitedVertices( # Loop over adjacent
                visit, visited): # vertices marking them as we visit them
                adj = j          # Next vertex is first adjacent unvisited
                break           # one, and the rest will be visited later
            if adj is not None: # If there's an adjacent unvisited vertex
                stack.push(adj) # Push it on stack and
                yield (adj, stack) # yield it with the path leading to it
```

```
else:                # Otherwise we're visiting a dead end so
    stack.pop()      # pop the vertex off the stack
```

The depth-first algorithm needs a stack that keeps track of the path of vertices as it traverses the graph. Python's `list` data type acts like a stack, including having a `pop()` method. Because it does not have a corresponding `push()` or `peek()` method, we define those in terms of equivalent operations in the simple `Stack` subclass of `list`. The four lines of Python code at the top of [Listing 14-4](#) show how little needs to be changed to implement a stack with a list.

The `depthFirst()` generator traverses the vertices using the rule-based approach outlined previously. After checking the index for the starting vertex, it makes a `visited` array filled with `False` for each of the `nVertices`. After the stack is created, the starting vertex, `n`, is pushed on it and marked as visited. The generator can now yield the first vertex, `n`, in the depth-first traversal.

The `depthFirst()` generator yields both the vertex index being visited and the stack (path) to the vertex because different callers need one or both of those. For example, to find the connected components of the graph you would need to collect only the vertices, whereas to solve a maze you would need the path that reaches the exit.

The main `while` loop of the `depthFirst()` generator applies the three rules. The top of the stack is the last vertex visited. It uses `peek()` to get its index and stores it in `visit`. The rules need to know if there are unvisited vertices adjacent to the `visit` vertex. The method assumes there are none by setting `adj` to `None` and calling `adjacentUnvisitedVertices()` generator to find such a vertex. If one is yielded, `adj` is updated with its index, and the inner `for` loop is exited. If none are found, `adj` remains `None`. The depth-first traversal needs only to find the first unvisited neighbor. The others will be found during a later pass through the loop.

Now we can test for Rule 1: *if there is an adjacent unvisited vertex, mark it as visited and push it on the stack*. The marking is done by `adjacentUnvisitedVertices()`, and `depthFirst()` pushes the adjacent vertex on the stack. The new vertex and its path are yielded to the caller for processing, and the `while` loop continues.

If Rule 1 doesn't apply, it now checks Rule 2: *if possible, pop a vertex off the stack*. At this point, the stack can't be empty because the `while` loop test checked that, and no vertices have been popped off since then. A simple `pop()` operation accomplishes Rule 2.

If neither Rule 1 nor Rule 2 applies, then you reach Rule 3: *you're done*. When the outer `while` loop finishes, all the reachable vertices have been pushed on the stack and then popped off, following the depth-first ordering.

You can test this code on a simple graph, as shown in [Figure 14-7](#).

```

graph = Graph()
graph.addVertex(Vertex('A'))
graph.addVertex(Vertex('B'))
graph.addVertex(Vertex('C'))
graph.addVertex(Vertex('D'))
graph.addVertex(Vertex('E'))
graph.addVertex(Vertex('F'))
graph.addVertex(Vertex('G'))
graph.addVertex(Vertex('H'))
graph.addVertex(Vertex('I'))
graph.addEdge(0, 1)
graph.addEdge(0, 2)
graph.addEdge(0, 3)
graph.addEdge(0, 4)
graph.addEdge(1, 5)
graph.addEdge(5, 7)
graph.addEdge(3, 6)
graph.addEdge(6, 8)
print('Depth-first traversal')
for vert, path in graph.depthFirst(0):
    print(graph.getVertex(vert).name,
          end='')
print()

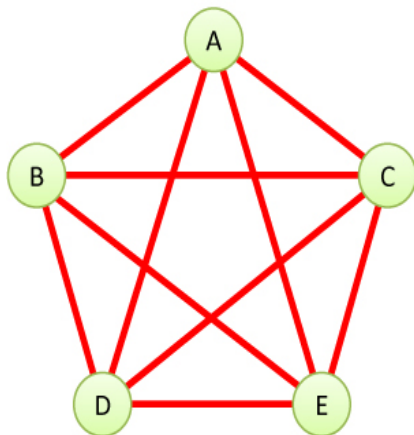
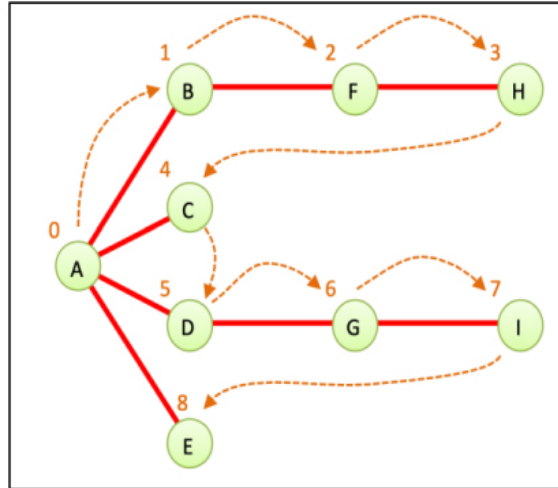
```

produces

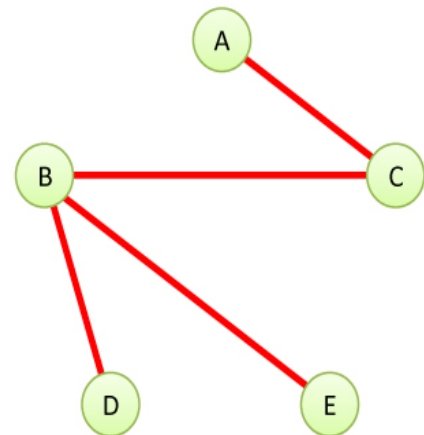
```

Depth-first traversal
ABFHCDGIE

```



A fully connected graph
(clique)



A minimum-spanning
tree

Figure 14-7 *A test of `depthFirst()` on a small graph*

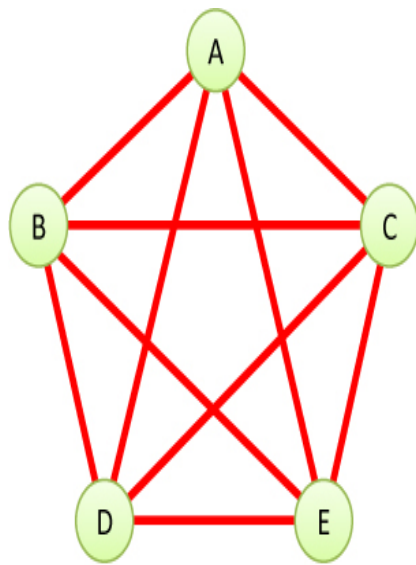
Looking back on the choice to yield both the vertex and the path to that vertex in the `depthFirst()` generator, note the redundancy because the vertex is always the last element in the path. You could rewrite the generator to return only the path, shifting the extraction of the vertex to the caller. That's not much of a change in Python where the caller can access the last vertex using an $O(1)$ reference to `path[-1]`. If the path is returned as a linked list, however, getting the last element takes $O(N)$ time and merits the extra return value.

Depth-First Traversal and Game Simulations

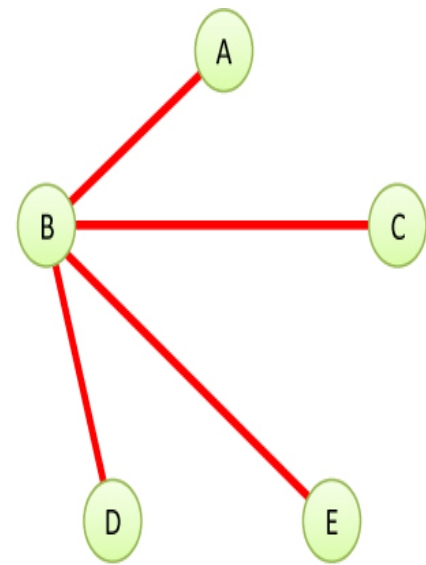
When you're implementing game programs, depth-first traversal can simulate the sequence of moves made by players. In two-player board games like chess, checkers, and backgammon, each player chooses from among a set of possible actions on their turn. The actions at each turn depend on the state of the game, sometimes called the *board state*. For example, in the initial state of chess, the actions are limited to movement of the pawns or knights. Subsequent board states allow actions involving the other pieces.

This behavior can be modeled with a graph where the possible actions form edges between board states, which are the vertices in a graph. Starting from a particular board state, depth-first traversal enumerates all possible board states that could be reached from the initial state.

Let's look at how that works for a simple game of tic-tac-toe. The first player, let's say it is the X-player, can make one of nine possible moves. The O-player can counter with one of eight possible moves, and so on. Each move leads to another group of choices by your opponent, which leads to another series of choices for you, until the last square is filled. [Figure 14-8](#) shows a partial graph of the board states. Only the edges connecting the topmost board state for each move are included, and symmetric board states are not drawn. That leaves only three distinct possible choices for the X-player's first move.



A fully connected graph
(clique)



Another minimum-spanning
tree

Figure 14-8 *Tic-tac-toe board states as a graph*

When you are deciding what move to make, one approach is to mentally imagine a move, then your opponent's possible responses, then your responses, and so on. You can decide what to do by seeing which move leads to the best outcome. In simple games like tic-tac-toe, the number of possible moves is sufficiently limited that it's possible to follow each path to the end of the game.

Analyzing these moves involves traversing the graph. As evident in [Figure 14-8](#), even a “simple” game like tic-tac-toe can lead to a large graph. The number of edges for the first move, after eliminating symmetric moves, is three. The O-player's first move includes four options. The next X-player move can have up to seven options after eliminating symmetry. The number of options diminishes after that.

Using depth-first traversal means the analysis drives toward the final states first. Any path that leads to an opponent's victory could be eliminated or, at least, set aside to be further explored as a last option. That would allow you to “prune” the graph significantly. How much could pruning save? If you don't pay attention to symmetry, there are nine possible first moves, followed by eight possible opponent moves, followed by seven possible first-player moves, and so on. That's $9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$ (9 factorial or $9!$ or 362,880) moves,

ignoring the reduction due to games ending after one player completes three in a row. Those moves are edges in the graph, and that's a lot of edges to follow. Pruning could make a huge savings.

Although exploring 362,880 edges seems manageable for modern computers, other games have much larger graphs. Chess has 64 squares and 16 pieces for each player. The game of go has a 19-by-19 grid of points where players place either a black or white stone. Those 361 points mean there are something like 361! potential sequences of moves (and that number doesn't account for the removal and replacement of stones). Even though some of the sequences result in the same board state, exploring the full graph is quite daunting. Most game-playing algorithms only explore the graph to a particular depth and use many techniques to eliminate as many paths as possible in analyzing move options. They may never generate the complete graph.

Breadth-First

The depth-first traversal algorithm acts as though it wants to get as far away from the starting point as quickly as possible. In the breadth-first, on the other hand, the algorithm likes to stay as close as possible to the starting point. It first visits all the vertices adjacent to the starting vertex, and only then goes further afield. This kind of traversal is implemented using a queue instead of a stack.

An Example

[Figure 14-9](#) shows the same graph as [Figure 14-5](#), but this time we traverse the graph breadth-first. Again, the numbers indicate the order in which the vertices are visited. Like before, A is the starting vertex. You mark it as visited and place it in an empty queue. Then you follow these rules:

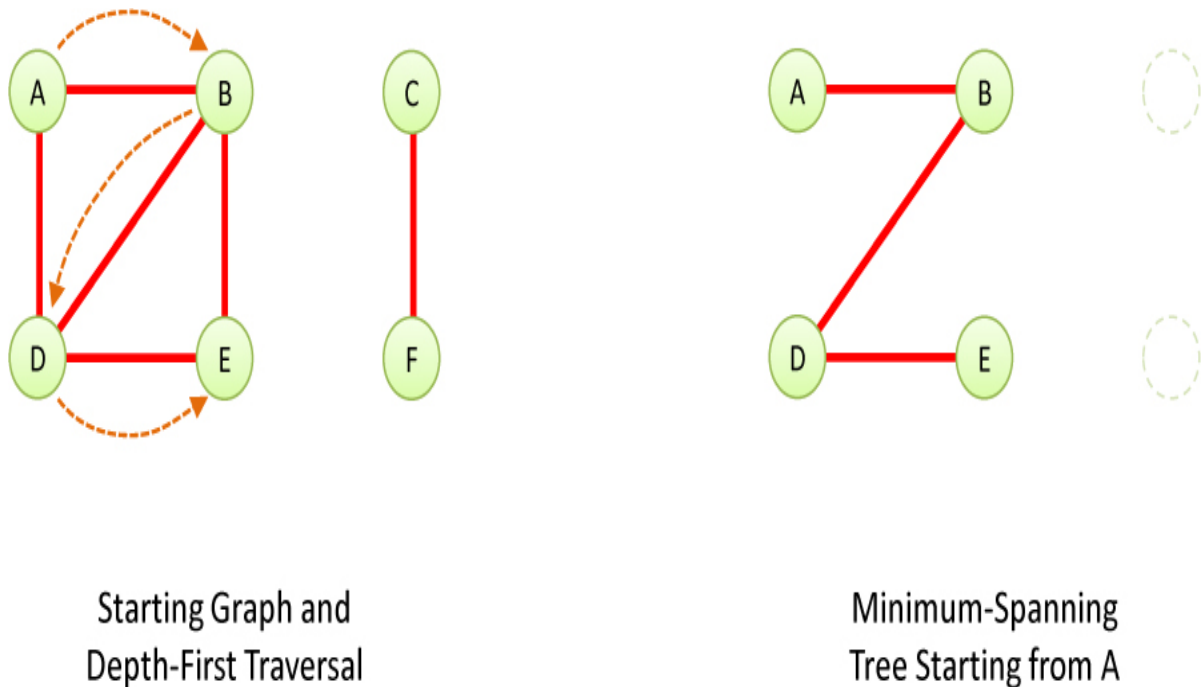


Figure 14-9 *Breadth-first traversal example*

Rule 1

Take the first vertex in the queue (if there is one) and insert all its adjacent unvisited vertices into the queue, marking them as visited.

Rule 2

If you can't carry out Rule 1 because the queue is empty, you're done.

The breadth-first traversal is slightly simpler than the depth-first traversal because there are only two rules. Walking through the example, you first visit A. Then you take all the vertices adjacent to A and insert each one into the queue as you visit and mark it. Now you've visited A, B, C, D, and E. At this point the queue (from front to rear) contains BCDE.

You apply Rule 1 again, removing B from the queue and looking for vertices adjacent to it. You find A and F, but A has been visited, so you visit and insert only F in the queue. Next, you remove C from the queue. It has no adjacent unvisited vertices, so nothing is visited or inserted in the queue. You remove D from the queue and find its neighbor G is unvisited, so you visit it and insert it in the queue. You remove E and find no unvisited neighbors.

At this point the queue contains FG, the only adjacent unvisited vertices found while previously visiting BCDE. You remove F and visit and insert H on the queue. Then you remove G and visit and insert I.

Now the queue contains HI. After you remove each of these and find no adjacent unvisited vertices, the queue is empty, so you're done. [Table 14-4](#) shows this sequence.

Table 14-4 *Queue Contents During Breadth-First Traversal*

Event	Queue (Front to Rear)
Visit A	A
Remove A	
Visit B	B
Visit C	BC
Visit D	BCD
Visit E	BCDE
Remove B	CDE
Visit F	CDEF
Remove C	DEF
Remove D	EF
Visit G	EFG
Remove E	FG
Remove F	G
Visit H	GH
Remove G	H
Visit I	HI
Remove H	I
Remove I	
Done	

At each moment, the queue contains the vertices that have been visited but whose neighbors have not yet been fully explored. (Contrast this breadth-first traversal with the depth-first traversal, where the contents of the stack hold the route you took from the starting point to the current vertex.) The nodes are visited by breadth-first in the order ABCDEFGHI.

The Graph Visualization Tool and Breadth-First Traversal

Use the Graph Visualization tool to try out a breadth-first traversal using the Breadth-First Traverse button. Again, you can experiment with the graph in [Figure 14-9](#), or you can make up your own.

Notice the similarities and the differences of the breadth-first traversal compared with the depth-first traversal.

You can think of the breadth-first traversal as proceeding like ripples widening when you drop a stone in water—or, for those of you who enjoy epidemiology, as the influenza virus carried by air travelers from city to city. First, all the vertices one edge away from the starting point (plane flight) are visited, then all the vertices two edges away are visited, and so on.

Python Code

The `breadthFirst()` method of the `Graph` class is like the `depthFirst()` method, except that it uses a queue instead of a stack and fully explores the sequence of adjacent unvisited vertices. The implementation for both the `Queue` class and the traversal generator is shown in [Listing 14-5](#).

Listing 14-5 *The `breadthFirst()` Traversal Generator for a Graph*

```
class Queue(list):          # Use list to define Queue class
    def insert(self, j): self.append(j) # insert == append
    def peek(self): return self[0] # First element is front of queue
    def remove(self): return self.pop(0) # Remove first element
    def isEmpty(self): return len(self) == 0

class Graph(object):
    ...
    def breadthFirst(
        self, n):          # Traverse the vertices in breadth-first
                           # order starting at vertex n
        self.validIndex(n) # Check that vertex n is valid
        visited = [False] * self.nVertices() # Nothing visited initially
        queue = Queue()    # Start with an empty queue and
        queue.insert(n)    # insert the starting vertex index on it
        visited[n] = True  # and mark starting vertex as visited
        while not queue.isEmpty(): # Loop until nothing left on queue
            visit = queue.remove() # Visit vertex at front of queue
            yield visit            # Yield vertex to visit it
            for j in self.adjacentUnvisitedVertices( # Loop over adjacent
```

```
visit, visited): # unvisited vertices
queue.insert(j) # and insert them in the queue
```

Here, we define the `Queue` class as a subclass of `list`. Inserting an item in the queue uses the list's `append()` method. This means the back (or end) of the queue is at the highest index of the list. That means, `peek()` and `remove()` operate on the first element of the list at index 0. The Python `list.pop()` method takes an optional parameter for the index of the item to remove.

The beginning of the traversal starts off just as it did for depth-first, checking the validity of the starting vertex and creating a `visited` array for all the vertices. Then the `visited` array is created and seeded with the starting vertex index, and that index is marked as visited. The implementation in [Listing 14-5](#) deviates slightly from the rules shown earlier in that the visit of the first vertex doesn't happen until inside the `while` loop where the rules are applied. Also note that marking vertices in the `visited` array happens before they are yielded to the caller. The overall `breadthFirst()` generator "visits" a vertex by yielding it to the caller for processing while keeping the internal `visited` array to track which vertices it has already put in its queue to process.

The `while` loop test determines whether Rule 1 or Rule 2 will apply. If it's Rule 1, the loop body removes a vertex from the front of the queue and immediately visits it by yielding it. The method visits the starting vertex in the same way all the others are visited. Note that we no longer have the stack to provide the path taken to reach this vertex. As a consequence, the `breadthFirst()` generator yields only a vertex index. It's possible and useful to provide the path, and we leave that as programming project.

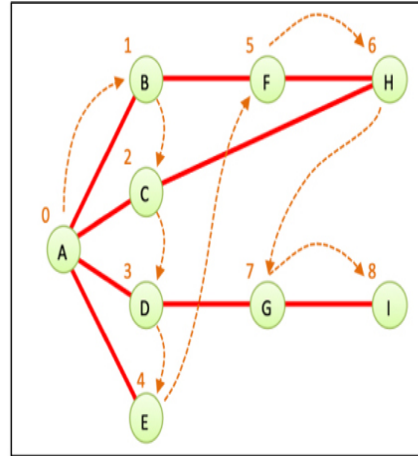
After `breadthFirst()` removes a vertex from the queue and visits it, the next thing to do is insert all the adjacent unvisited vertices in the queue. That process is handled by looping through the `adjacentUnvisitedVertices()` generator (shown in [Listing 14-3](#)) and inserting the vertices in the queue. That completes the implementation of Rule 1. Rule 2 is also done because no more processing is needed when the queue is empty.

Let's test the `breadthFirst()` traversal on nearly the same graph used for depth-first, but let's add one edge between C and H to see what effect it has. [Figure 14-10](#) shows the code used to construct the graph and traverse the vertices in breadth-first order. The extra edge is highlighted in the code. Note that the vertex ID numbers used to create the edges are different from the visit numbers next to the vertices in the figure.

```

graph = Graph()
graph.addVertex(Vertex('A'))
graph.addVertex(Vertex('B'))
graph.addVertex(Vertex('C'))
graph.addVertex(Vertex('D'))
graph.addVertex(Vertex('E'))
graph.addVertex(Vertex('F'))
graph.addVertex(Vertex('G'))
graph.addVertex(Vertex('H'))
graph.addVertex(Vertex('I'))
graph.addEdge(0, 1)
graph.addEdge(0, 2)
graph.addEdge(0, 3)
graph.addEdge(0, 4)
graph.addEdge(1, 5)
graph.addEdge(5, 7)
graph.addEdge(3, 6)
graph.addEdge(6, 8)
graph.addEdge(2, 7)
print('Breadth-first traversal')
for vert in graph.breadthFirst(0):
    print(graph.getVertex(vert).name,
          end='')
print()

```



produces
Breadth-first traversal
ABCDEFHGI

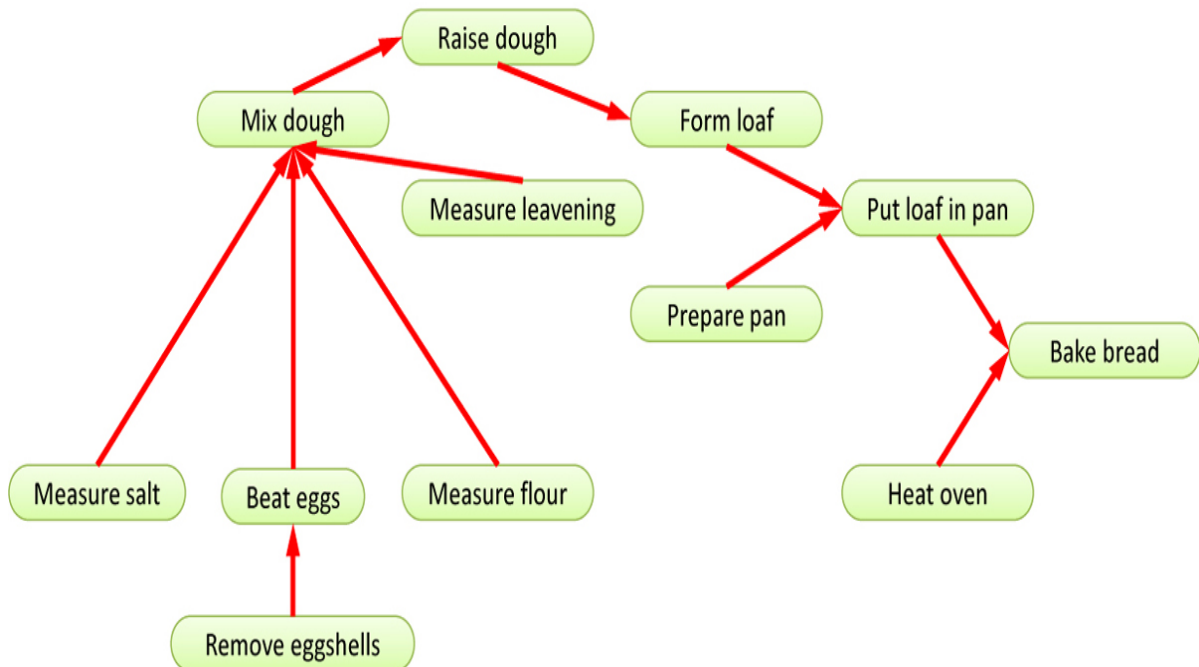


Figure 14-10 A test of `breadthFirst()` on a small graph

In the test example, after visiting ABCDE, the queue contains the unvisited adjacent vertices, FHG. Without the edge from C to H, the queue would have been FG, like in the graph of [Figure 14-9](#). With the edge CH, H is marked as visited and inserted on the queue when C is removed. Then F is removed, and its neighbors searched for unvisited vertices. H has not yet been visited (yielded by the generator that starts at F), but it was marked as visited as it was added to the queue. So, H does not show up as a vertex to add to the queue. The next pass through the `while` loop removes H from the front of the queue and visits it. Because it, too, has no adjacent unvisited neighbors, nothing is put on the queue. The next pass removes G from the queue, finds I as unvisited, and inserts that final vertex.

This example illustrates an interesting property of breadth-first traversal and search: it first finds all the vertices that are one edge away from the starting point, then all the vertices that are two edges away, and so on. This capability is useful if you're trying to find the *shortest path* from the starting vertex to a given vertex (see Project 14.3). You start a breadth-first traversal, and when you visit a particular vertex, you know the path you've traced so far is the shortest path to it. If there were a shorter path, the breadth-first traversal would have visited it already. In [Figure 14-10](#), there are two paths from A to H: ABFH and ACH. The fact that breadth-first visits H before G is due to having a two-edge path using the added edge. In [Figure 14-9](#), the absence of edge CH means G is visited before H because both G and H can only be reached through three-edge paths.

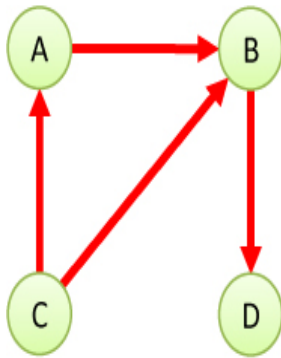
Minimum Spanning Trees

Suppose you're designing a large apartment building and need to decide where the hot water pipes should go. The example shown in [Figure 14-4](#) shows a completed plumbing network for a two-and-a-half-bathroom house, but let's imagine you have hundreds of sinks, bathtubs, showers, washing machines, and hot water heaters to connect. For the water supply, you could connect every fixture to every other one with a pipe. That would certainly provide the shortest possible path between every pair of fixtures. That would also cost a lot, and any plumber asked to do the installation would probably laugh.

After you get over your embarrassment, you realize that you only really need to connect every sink, bathtub, shower, and washing machine to a water heater.

You want those pipes to be short so that the hot water can reach the faucets quickly and to minimize the length of pipe needing insulation. You can't run the pipes through the rooms of the building; they must be hidden inside the walls, floors, and ceilings. Not every wall will allow pipes to go through it, such as a wall made of glass. So, you must take some twisted paths to hook everything together. How do you find which paths to hook up so that all the fixtures are connected but the amount of pipe is the shortest?

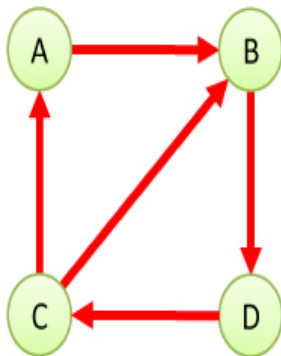
It would be nice to have an algorithm that, for any connected set of fixtures and pipes (vertices and edges, in graph terminology), would find the minimum set of pipes needed to connect all the fixtures. Even if it didn't find the absolute minimum pipe length overall or the minimum total distance to the water heater, you wouldn't want any extra pipes connecting two fixtures if there was another path between them. For instance, you wouldn't want a loop of pipes (edges) like the path ABFHCA in [Figure 14-10](#) or the fully connected graph on the left of [Figure 14-11](#) (although there are cases where plumbing loops are desirable). The result of such an algorithm would be a graph with the minimum number of edges necessary to connect the vertices. The graph on the left of [Figure 14-11](#) has the maximum number of edges, 10, for a five-vertex graph. The graph on the right has the same five vertices but with the minimum number of edges necessary to connect them, four. This constitutes a **minimum spanning tree (MST)** for the graph.



	A	B	C	D
A	0	1	0	0
B	0	0	0	1
C	1	1	0	0
D	0	0	0	0

Acyclic

Adjacency Matrix



	A	B	C	D
A	0	1	0	0
B	0	0	0	1
C	1	1	0	0
D	0	0	1	0

Cyclic

Adjacency Matrix

Figure 14-11 *A fully connected graph and a minimum spanning tree*

There are many possible minimum spanning trees for a given graph. The MST of [Figure 14-11](#) shows edges AC, BC, BD, and BE, but edges AC, CE, ED, and DB would do just as well. The arithmetically inclined will note that the number of edges in a minimum spanning tree is always one fewer than the number of

vertices. Removing any edge from a minimum spanning tree would create multiple connected components.

For now, don't worry about the length of the edges. You're not trying to find a minimum physical length, just the minimum number of edges. (Your plumber might have a different opinion). We discuss more about this when we talk about weighted graphs in the next chapter.

The algorithm for creating the minimum spanning tree is almost identical to that used for traversing. It can be based on either the depth-first or the breadth-first traversal. This example uses the depth-first traversal.

Perhaps surprisingly, by executing the depth-first traversal and recording the edges you've traveled to reach a vertex, you automatically create a minimum spanning tree. It's also somewhat counterintuitive because breadth-first traversal finds the shortest path to each vertex. The only difference between the minimum spanning tree method, which you see in a moment, and the depth-first traversal, which you saw earlier, is that it must somehow record and return a tree-like structure.

Minimum Spanning Trees in the Graph Visualization Tool

The Graph Visualization tool runs the minimum spanning tree algorithm after you select a starting vertex and select the corresponding button. You see the `depthFirst()` generator run in the code window to yield results to a method we explore shortly.

As the depth-first traversal yields vertices and their paths from the starting vertex, the Visualization tool highlights them, as shown in [Figure 14-12](#). In the example, the algorithm adds vertex E and its path from the starting vertex, C, to the tree it is building. The vertices that have been added have brown circles around them (B, C, D, and E) and the edges forming the path returned by `depthFirst()` are highlighted in blue. Edges added in earlier steps (for example, edge BD) are highlighted in brown.

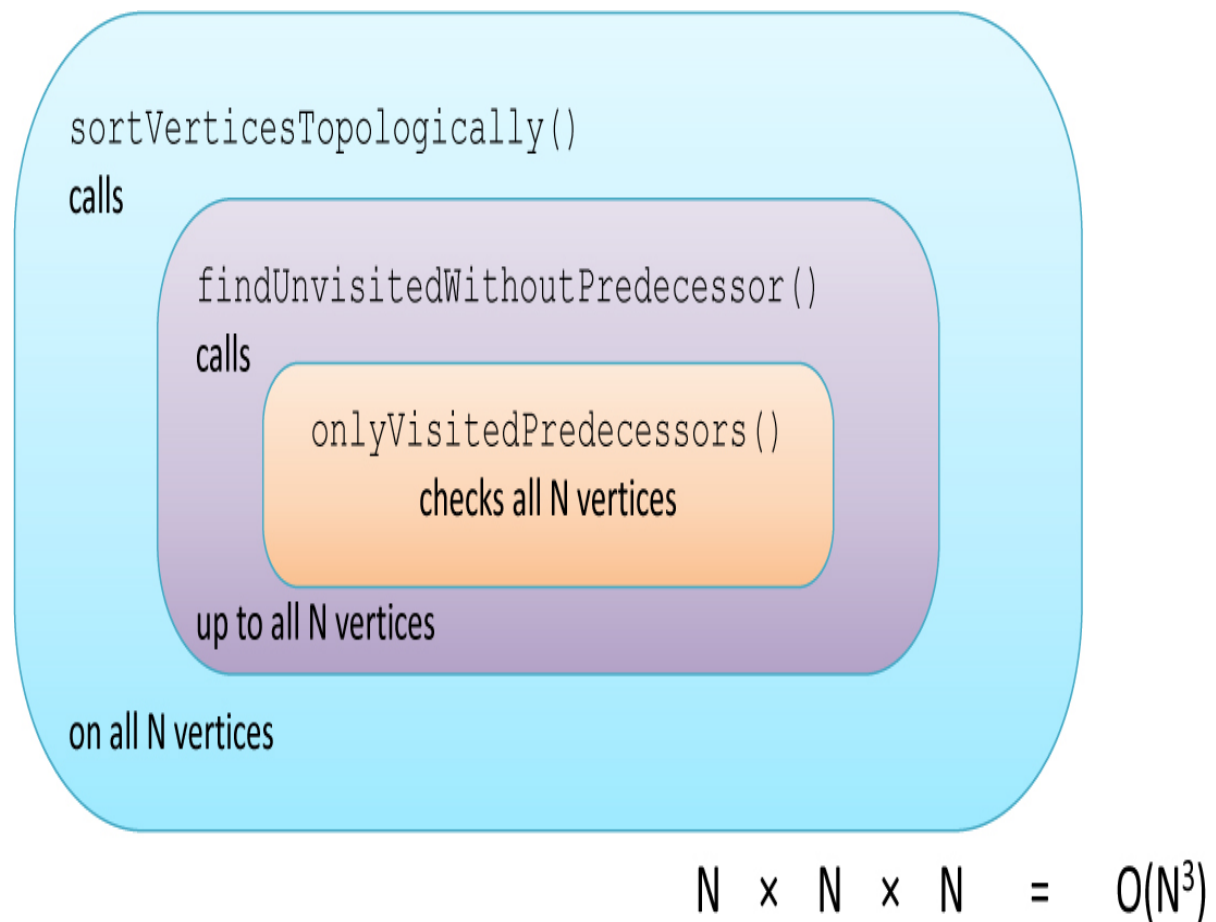


Figure 14-12 *The Graph Visualization tool adding a path to a minimum spanning tree*

As you can see, the minimum spanning tree implementation must track many things as it runs. We explore the detailed implementation in Python, but first we need to revisit a structure that we discussed in detail in several preceding chapters: trees.

Trees Within a Graph

What does a minimum spanning tree “look” like? Is it a binary tree? No, because the number of vertices connected to a given vertex could be one, two, three, and so on. For example, in the minimum spanning tree on the right of [Figure 14-13](#), there is no way to arrange the vertices and edges into a binary tree form. Even though the trees for both [Figure 14-11](#) and [Figure 14-13](#) come from the same starting graph, it’s not possible to know whether the resulting tree will have edges that could form a binary tree.